

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Annexure A

Coding challenge

20
20
20
Total Marks: 25

Code of Conduct:

I P. Greeshma (192424418) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

cy

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.
4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class `StarSwitch` with methods `add_port(host_id)`, `remove_port(host_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

Q1: Star Topology

```
def generate_star_topology(n_hosts):
```

```
    switch = "SW1"
```

```
    print("\n" + "=" * 50)
    print("                STAR TOPOLOGY")
    print("=" * 50)
```

```
    print("\n
    print("                / | | \\", switch)
```

```
    for i in range(1, n_hosts + 1):
```

```
        host = f"Host{i}"
        length = 10 * i
```

```
        print(f"{host:<10} ---- Cat6 UTP ({length} m) ---- {switch}")
```

```
    print("\nCentral Switch :", switch)
    print("Cable Type       : Cat6 UTP")
    print("Maximum Length : 100 m")
```

Input

```
n = int(input("Enter number of hosts: "))
```

```
if n > 0:
```

```
    generate_star_topology(n)
```

```
else:
```

```
    print("Enter a positive number.")
```

Enter number of hosts: 4

=====

STAR TOPOLOGY

=====

```

          SW1
        /  |  |  \
Host1  ---- Cat6 UTP (10 m) ---- SW1
Host2  ---- Cat6 UTP (20 m) ---- SW1
Host3  ---- Cat6 UTP (30 m) ---- SW1
Host4  ---- Cat6 UTP (40 m) ---- SW1
```

```
Central Switch : SW1
Cable Type     : Cat6 UTP
Maximum Length : 100 m
```


Q2: Validate UTP Segment Length

```
def validate_segment(length_m):  
    if length_m < 0:  
        return "Error: Length cannot be negative"  
  
    elif length_m <= 100:  
        return True  
  
    else:  
        return "Error: UTP segment exceeds 100 m"
```

Test Cases

```
test_cases = [50, 100, 120]
```

```
print("UTP SEGMENT VALIDATION")  
print("-" * 30)
```

```
for length in test_cases:
```

```
    result = validate_segment(length)
```

```
    print(f"Length: {length} m")  
    print("Result:", result)  
    print()
```

UTP SEGMENT VALIDATION

Length: 50 m
Result: True

Length: 100 m
Result: True

Length: 120 m
Result: Error: UTP segment exceeds 100 m

Q3: CSMA/CD Simulation

```
import random
```

```
LOG_FILE = "csma_log.txt"
```

```
hosts = ["Host1", "Host2", "Host3", "Host4"]
```

```
print("=" * 50)  
print("                CSMA/CD SIMULATION")  
print("=" * 50)
```



```

with open(LOG_FILE, "w") as log:
    for attempt in range(1, 6):
        sender = random.choice(hosts)
        message = f"Attempt {attempt}: {sender} wants to transmit"
        print(message)
        log.write(message + "\n")
        collision = random.choice([True, False])
        if collision:
            print("Collision detected!")
            log.write("Collision detected!\n")
            backoff = random.randint(1, 10)
            message = (
                f"{sender} back-off interval = "
                f"{backoff} time units"
            )
            print(message)
            log.write(message + "\n")
        else:
            print(f"{sender} transmitted successfully")
            log.write(
                f"{sender} transmitted successfully\n"
            )
        print()

print("=" * 50)
print("Simulation completed")
print("Log file created:", LOG_FILE)
print("=" * 50)

```

CSMA/CD SIMULATION

Attempt 1: Host1 wants to transmit
Host1 transmitted successfully

Attempt 2: Host4 wants to transmit
Collision detected!
Host4 back-off interval = 10 time units

Attempt 3: Host4 wants to transmit
Host4 transmitted successfully

Attempt 4: Host4 wants to transmit
Collision detected!
Host4 back-off interval = 4 time units

Attempt 5: Host4 wants to transmit
Host4 transmitted successfully

```
=====
Simulation completed
Log file created: csma_log.txt
=====
```

Q4: Star Switch Communication Simulation
Google Colab friendly version

```
class StarSwitch:
```

```
    def __init__(self):
```

```
        self.mac_table = {}
```

```
    def connect_host(self, host, mac, port):
```

```
        self.mac_table[mac] = {
            "host": host,
            "port": port
        }
```

```
        print(f"{host} connected to Port {port}")
```

```
    def transmit_frame(self, source_mac, destination_mac, message):
```

```
        print("\n" + "=" * 50)
        print("FRAME TRANSMISSION")
        print("=" * 50)
```

```
        print("Source MAC      :", source_mac)
        print("Destination MAC :", destination_mac)
        print("Message          :", message)
```

```
        # Learn source MAC
```

```
        if source_mac in self.mac_table:
```

```
            source_host = self.mac_table[source_mac]["host"]
```



```
# Frame 1: Host1 -> Host2
switch.transmit_frame(
    "AA:AA:AA:AA:AA:01",
    "BB:BB:BB:BB:BB:02",
    "Hello Host2"
)
```

```
# Frame 2: Host2 -> Host1
switch.transmit_frame(
    "BB:BB:BB:BB:BB:02",
    "AA:AA:AA:AA:AA:01",
    "Hello Host1"
)
```

Host1 connected to Port Port 1
Host2 connected to Port Port 2

=====

FRAME TRANSMISSION

=====

Source MAC : AA:AA:AA:AA:AA:01
Destination MAC : BB:BB:BB:BB:BB:02
Message : Hello Host2

Source Host: Host1
Frame forwarded to Host2 through Port 2

MAC ADDRESS TABLE

AA:AA:AA:AA:AA:01 -> Host1 -> Port 1
BB:BB:BB:BB:BB:02 -> Host2 -> Port 2

=====

FRAME TRANSMISSION

=====

Source MAC : BB:BB:BB:BB:BB:02
Destination MAC : AA:AA:AA:AA:AA:01
Message : Hello Host1

Source Host: Host2
Frame forwarded to Host1 through Port 1

MAC ADDRESS TABLE

AA:AA:AA:AA:AA:01 -> Host1 -> Port 1
BB:BB:BB:BB:BB:02 -> Host2 -> Port 2

Q5: StarSwitch Class

class StarSwitch:


```
for host in self.ports:
    if host != source:
        print(
            f"{source} --> {host}"
        )
```

```
# Create switch
switch = StarSwitch()
```

```
# Add ports
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")
```

```
# Create frame
frame = {
    "source": "Host1",
    "message": "Hello Network"
}
```

```
# Broadcast frame
switch.broadcast(frame)
```

```
# Remove a port
print()
switch.remove_port("Host4")
```

```
# Display remaining ports
print("\nConnected Hosts:")
print(switch.ports)
```

```
Host1 connected to SW1
Host2 connected to SW1
Host3 connected to SW1
Host4 connected to SW1
```

```
=====
BROADCAST FRAME
=====
```

```
Source : Host1
Message : Hello Network
```

```
Frame Delivery:
Host1 --> Host2
Host1 --> Host3
Host1 --> Host4
```